



Formation DevOps ▪ SRE ▪ Infra

Claude Code pour les Ops

Piloter un agent de coding en environnement bancaire

3 jours ▪ 10 modules ▪ 12 labs ▪ 1 capstone

Pourquoi cette formation

Le constat

- L'IA générative est arrivée dans les équipes Dev.
- Côté **exploitation**, l'usage reste artisanal : des prompts jetés, non partagés, non tracés.
- Or c'est là que le risque est le plus élevé : l'agent touche à de l'**infrastructure réelle**.

Le parti pris

- On n'apprend pas « à prompter ».
- On apprend à **outiller** et à **cadrer** un agent.
- Chaque brique est posée quand sa nécessité se fait sentir.
- Chaque garde-fou est **testé**, pas décrit.

À retenir

L'objectif n'est pas de coder plus vite. C'est de transformer vos **procédures d'équipe** en actifs exécutables, versionnés et auditable.

À qui s'adresse la formation

Public

- Ingénieurs **DevOps, SRE, Cloud / IaC, Infra, Exploitation**.
- **Débutants acceptés** sur Claude Code et sur l'IA générative.
- Aucun pré-requis en développement applicatif.

Pré-requis réels

- Être à l'aise avec un terminal Linux / macOS.
- Connaître Git.
- Maîtriser au moins une techno d'infra : Docker, Kubernetes, Terraform ou Ansible.

Format

≈ **30 % de théorie / 70 % de pratique**. Tout est déroulé **en direct dans un terminal**, sur une application microservices réelle.

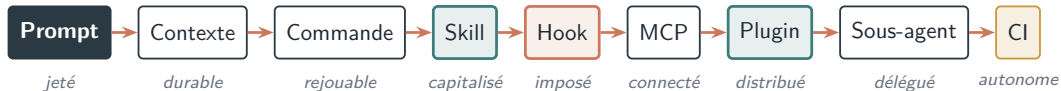
Ce que vous saurez faire à la fin

1. Piloter une session et ses **commandes natives** — cadrer, vérifier, revenir en arrière.
2. Donner à l'agent un **contexte durable** et fiable, partagé par toute l'équipe.
3. Transformer une procédure d'équipe en **skill** rejouable et versionné.
4. Imposer des **garde-fous déterministes** : anti-secret, blocage destructif, journal d'audit.
5. Connecter l'agent à GitHub, Terraform, Grafana via **MCP** — officiels puis internes.
6. **Packager et distribuer** un toolkit d'équipe via un marketplace interne.
7. Déléguer à des **sous-agents** spécialisés : revue de plan, diagnostic de pod.
8. Faire tourner l'agent **sans humain** : mode headless, boucles, intégration CI.
9. Dérouler une chaîne complète *provisionner* → *déployer* → *observer* → *sécuriser*.

La progression pédagogique



Le fil conducteur : chaque brique lève une limite



À retenir

On ne passe à la brique suivante qu'après avoir **senti la limite** de la précédente. Chaque module s'ouvre sur cette limite, en une phrase.

Les 10 modules — la limite que chacun lève

#	Module	La limite qu'il lève
00	Prise en main	« Par où je commence ? »
01	Commandes natives	« Je ne sais pas ce que l'outil sait faire. »
02	Contexte & mémoire	« Il refait toujours la même erreur. »
03	Commandes personnalisées	« Je retape le même prompt tous les jours. »
04	Skills	« Ma procédure fait quinze étapes. »
05	Hooks	« Et s'il lance un terraform destroy ? »
06	MCP	« Il ne voit ni nos tickets ni nos métriques. »
07	Plugins	« Comment je donne tout ça à mes douze collègues ? »
08	Sous-agents	« Ma session est noyée sous 4 000 lignes de logs. »
09	Automatisation & CI	« Et sans moi devant le clavier ? »

Les 12 labs — vos cas d'usage

Bloc 1 — Code & collaboration

- 01 Git & GitHub assistés
- 02 Pipeline CI/CD avec gate manuel
- 03 Tests automatisés

Bloc 2 — Garde-fous

- 04 Sécurité, secrets & audit
- Posé *avant* tous les labs d'infrastructure.

Bloc 3 — Provisionnement & IaC

- 05 VM Multipass reproductibles
- 06 Terraform & revue de plan
- 07 Ansible & durcissement

Bloc 4 — Conteneurs & orchestration

- 08 Conteneurisation & scan d'images
- 09 Cluster Kubernetes & diagnostic
- 10 Packaging Helm multi-environnements

Bloc 5 — Run & fiabilité

- 11 Observabilité & alerting
- 12 Incident & post-mortem

Capstone

La chaîne complète, de bout en bout, en binômes — avec épreuve croisée.

Les labs se déroulent **dans l'ordre**, de 01 à 12.

Déroulé des 3 jours

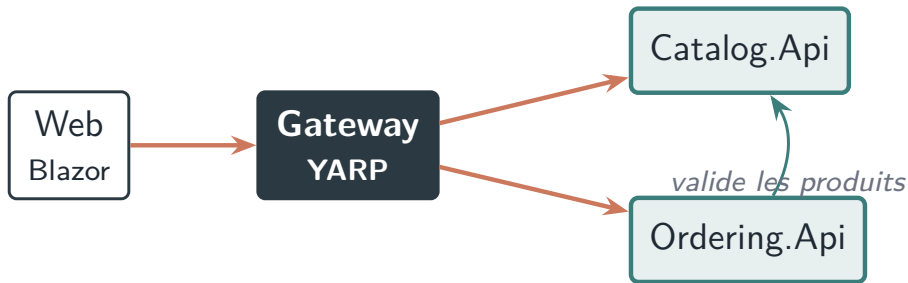


Jour	Matin	Après-midi
J1 <i>Fondations & outil</i>	Fondations IA générative & prompt engineering Ops Atelier prompting	Modules 00 → 04 session, commandes, contexte, skills
J2 <i>Garde-fous & premiers cas</i>	Module 05 (hooks) Lab 01 (Git) Modules 06 et 07	Modules 08 et 09 Labs 02 → 04 (CI/CD, tests, sécurité)
J3 <i>Infrastructure & run</i>	Labs 05 → 08 (Multipass, Terraform, Ansible, Docker)	Labs 09 → 12 Capstone + restitution

À retenir

Les **dix modules** d'abord (l'outil), puis les **douze labs** (le métier). Variables d'ajustement en cas de retard : les labs **03**, **10** et **11**. Jamais le lab **04** ni les garde-fous.

L'application fil rouge



Ce que c'est

Microservices **.NET / ASP.NET Core**, orchestrés par **.NET Aspire**. OpenTelemetry, health checks et résilience HTTP déjà en place.

Livrée volontairement nue

Ni `.claude/`, ni `Dockerfile`, ni manifests, ni CI. **C'est vous qui les produisez** au fil des labs.

Le cadre : garde-fous & conformité



Un garde-fou qui dépend du modèle n'est pas un garde-fou.

Niveau	Mécanisme	Fiabilité	Usage
1	Consigne dans le prompt	Aléatoire	Style, préférences
2	Règle dans <code>CLAUDE.md</code>	Bonne, non garantie	Conventions d'équipe
3	Règle <code>/permissions</code>	Appliquée par le harnais	Cadrer les outils
4	Hook	Déterministe	Garde-fous & audit
5	Contrôle plateforme (gate CI, RBAC)	Hors périmètre de l'agent	Production

À retenir

Plus l'action est irréversible, plus le niveau doit être élevé. Un déploiement en production se protège au niveau 5 — jamais au niveau 1.

Les six règles du cadre bancaire

	Règle	Mise en œuvre technique
R1	Aucune donnée sensible dans un prompt	Hook anti-secret, gitleaks, <code>.gitignore</code>
R2	Aucune action destructive sans validation	Hook guard-destructive + <code>/permissions</code>
R3	Aucun déploiement prod automatique	Gate manuel, environnement protégé côté forge
R4	Tout l'IaC généré passe en revue humaine	Sous-agent tf-plan-reviewer, mode plan
R5	Traçabilité des actions de l'agent	Hook d'audit : journal horodaté JSON Lines
R6	Une donnée externe n'est pas une instruction	MCP en lecture seule, scopes minimaux

Le principe qui prime sur tous les autres

L'IA propose, l'ingénieur décide et teste. Un hook n'est pas un modèle : il s'exécute toujours.

Ce qu'on ne fait pas pendant ces 3 jours

- Aucune connexion à un **système de production**.
- Aucun **jeton d'entreprise** : comptes et dépôts de test personnels uniquement.
- Aucune **donnée réelle** : l'application ne contient que des données fictives.
- Aucun apply sur une infrastructure partagée — Terraform cible le provider local, Kubernetes tourne sur k3d.

Et pourtant, tout est réel

Les images sont réellement construites et scannées, le cluster tourne, les pods plantent pour de vrai, et les hooks bloquent réellement vos commandes. **Aucune simulation.**

Pratique



Vérifier son poste — maintenant

```
# 1. Claude Code
curl -fsSL https://claude.ai/install.sh | bash
claude --version && claude doctor

# 2. L'outillage de la formation
./docs/check-prereqs.sh

# 3. L'application fil rouge
cd ecommerce-app && dotnet build ECommerce.slnx
```

Indispensable

claude, git, jq, gh, dotnet

Selon les labs

docker, multipass, terraform, ansible,
kubectl, k3d, helm, gitleaks, trivy

Pour vous

- Le dépôt complet : modules, labs, corrigés.
- La **cheat-sheet** commandes & préfixes.
- Le document **garde-fous & conformité**.

Pour votre équipe

- Une bibliothèque de **skills** Ops.
- Les **hooks** de garde, testés.
- Le **plugin** d'équipe, installable.
- Votre plan **30 / 60 / 90 jours**.

À retenir

Le vrai livrable n'est pas ce dépôt : c'est ce que vous mettrez en place **la semaine prochaine**, sur un dépôt réel.

On commence.

Module 00 — Prise en main